Intelligent Assets

Autonomous Financial Instruments
with Embedded Compliance

Raeez Lorgat
Momentum
raeez@momentum.inc

March 2026 v2.0

Abstract

We develop a theory of *intelligent assets*—financial instruments that carry executable programs and a compliance function as intrinsic properties. The programs enable autonomous operation: an intelligent asset can rebalance a portfolio, negotiate trades, migrate between jurisdictions, and file regulatory reports without human intervention. The compliance function—a map from twenty regulatory domains to a five-element ordered set—ensures that every autonomous action is evaluated against applicable regulations before execution. We show that the compliance function is fail-closed by construction and that sanctions violations are algebraically non-overridable. Cross-jurisdictional compliance composes through lattice operations, enabling an asset to accumulate sovereign attestations as it moves between jurisdictions. We describe the execution architecture, trace a worked example through the lifecycle of an autonomous private equity fund, and identify the hard open problems—including the oracle problem (evaluator correctness is unverifiable from within the system) and agent alignment (who governs the asset's bound programs).

Contents

1 Introduction **2**

2 Architecture **3**

3 The Compliance Lattice **4**

4 Execution **5**
4.1 The Virtual Machine 5
4.2 The Compliance Gate 5
4.3 Receipts 6
4.4 Triggers 6
4.5 Negotiation 6

5 Composition **6**

6 Worked Example: An Autonomous PE Fund **7**

7 Comparison **8**
7.1 Smart Contracts 8
7.2 Traditional Compliance 8

8 Limitations **9**

9 Conclusion **9**

1 Introduction

A financial instrument—a fund, a bond, a structured product—carries programs that let it act on its own: rebalance a portfolio when allocations drift, file a regulatory report at quarter-end, migrate to a new jurisdiction when terms improve. Before each action, a compliance function evaluates the action against every applicable regulatory domain. If the evaluation passes, the action executes. If it fails, the action halts and the failure is recorded. The compliance verdicts are signed by sovereign evaluators and travel with the asset as portable attestations.

The programs make the asset autonomous. The compliance function makes it legal.

Consider a private equity fund domiciled in Abu Dhabi. It holds positions across MENA real estate and Southeast Asian infrastructure debt. Its rebalancing program monitors portfolio concentration. When MENA allocation exceeds a threshold, the program initiates a rotation: it identifies counterparty assets, evaluates twenty regulatory domains—anti-money laundering, sanctions, securities classification, tax, custody, cross-border capital controls—and, if every domain clears, executes the trade. If the fund later identifies better terms in Singapore, a migration program evaluates exit requirements, presents the fund's attestation portfolio to Singapore's regulator, obtains fresh clearances where required, and completes the move. Rebalancing or migration takes seconds.

This is not how cross-border finance works today. Autonomous operation across jurisdictions is not technically difficult; it is regulatorily impossible. A fund rebalancing across three jurisdictions needs three law firms, three compliance teams, and months of manual process. Global compliance costs exceed $270 billion annually.[1] The real loss is the financial instruments that cannot exist because compliance overhead makes autonomous cross-border operation uneconomic.

The missing piece is a compliance primitive that is *intrinsic* to the asset (not maintained by an external institution), *machine-executable* (evaluable in microseconds), *sovereignly attested* (carrying legal authority), and *composable* across jurisdictions (combinable through algebraic operations). This paper provides that primitive.

2 Architecture

Definition 2.1 (Intelligent Asset)**.** An *intelligent asset* $A = (g, \pi, \sigma, s, P)$ consists of:

- g: an immutable *genesis document* recording the asset's type, issuing jurisdiction, initial parameters, and content-addressed identity.
- π: an *attestation portfolio*—the set of sovereign-issued compliance verdicts accumulated over the asset's lifetime.

[1] Thomson Reuters, "Cost of Compliance," 2024.

- σ: a *receipt chain*—a hash-linked sequence of execution records.
- $s : W \to W$: a persistent key-value store (32-byte keys and values) that survives across program executions.
- $P = \{p_1, \ldots, p_k\}$: a set of *bound programs*—bytecode executed by the asset's virtual machine.

Without P, the asset is a record with signed metadata. With P, the asset observes external events, makes decisions, and acts—subject to a compliance gate described in §3. The five-tuple is what separates an intelligent asset from a ledger entry.

Definition 2.2 (Jurisdiction)**.** A *jurisdiction* $J = (E_J, L_J, K_J)$ is a compliance evaluation service: E_J is an evaluator function, L_J is a content-addressed set of regulatory rules (a *lawpack*), and K_J is a signing key. The jurisdiction evaluates an asset's intended action against its rules and, if the action is compliant, signs an attestation.

The jurisdiction does not execute the asset's programs. It certifies compliance; the asset executes only upon certification.

Definition 2.3 (Compliance Domain)**.** The set of compliance domains $\mathcal{D} = \{d_1, \ldots, d_{20}\}$ is fixed at twenty: anti-money laundering, know-your-customer, sanctions, tax, securities, corporate governance, custody, data privacy, licensing, digital assets, consumer protection, environmental and social, cross-border, reporting, insurance, banking, payment, investment, intellectual property, dispute resolution.

Every domain must be explicitly addressed for every operation. Adding or removing a domain is a breaking change that forces review of every evaluator, every corridor, and every bound program. The set is exhaustive by design: there is no regulatory surface an asset can touch without evaluation.

Definition 2.4 (Attestation)**.** An *attestation* $\alpha = (J, d, v, t_{\text{issue}}, t_{\text{expire}}, \sigma_J)$ records that jurisdiction J evaluated domain d and assigned compliance state v, valid from t_{issue} to t_{expire}, signed with K_J.

Attestations are W3C Verifiable Credentials signed with Ed25519. They expire by design, forcing periodic re-evaluation. They are revocable, because regulations change and errors must be correctable. The trust model is explicit and scoped: accepting an attestation means trusting the issuer's evaluator for the attested domain under the attested lawpack version.

Definition 2.5 (Corridor)**.** A *corridor* $C(J_1, J_2) = (R, \varphi)$ between jurisdictions specifies a *re-evaluation set* $R \subseteq \mathcal{D}$ (domains requiring fresh evaluation by J_2) and a *domain mapping* φ (regulatory equivalences between J_1 and J_2). Corridors are asymmetric: $C(J_1, J_2) \neq C(J_2, J_1)$ in general.

3 The Compliance Lattice

Every domain of an intelligent asset is in one of five states, totally ordered:

$$\textsc{NonCompliant} < \textsc{Pending} < \textsc{Exempt} < \textsc{Compliant} < \textsc{NotApplicable}$$

This is a bounded lattice with $\wedge = \min$, $\vee = \max$, bottom $\bot = \textsc{NonCompliant}$, top $\top = \textsc{NotApplicable}$. The *passing* states are $\{\textsc{Exempt}, \textsc{Compliant}, \textsc{NotApplicable}\}$—everything strictly above $\textsc{Pending}$.

Definition 3.1 (Compliance Tensor)**.** The *compliance tensor*[2] $T_J(A) : \mathcal{D} \to \mathcal{S}$ assigns each of the twenty domains a compliance state, parameterized by jurisdiction J and asset A.

Definition 3.2 (Verdict)**.** The *verdict* of tensor T with respect to an operation requiring domains $D_{\text{req}} \subseteq \mathcal{D}$ is $V(T) = \bigwedge_{d \in D_{\text{req}}} T[d]$. The asset may act if and only if $V(T)$ is passing.

The verdict is operation-specific. A domestic portfolio rebalancing might require $D_{\text{req}} = \{\text{AML}, \text{KYC}, \text{Sanctions}, \text{Securities}\}$. A cross-border migration requires all twenty.

Two properties follow from the lattice structure.

Proposition 3.1 (Sanctions Non-Overridability)**.** If $T[\text{SANCTIONS}] = \text{NONCOMPLIANT}$ and $\text{SANCTIONS} \in D_{\text{req}}$, then $V(T) = \text{NONCOMPLIANT}$ regardless of all other domains.

Proof. $\bot \wedge x = \bot$ for all x. □

No combination of passing attestations in other domains can override a sanctions violation. This is a structural property of the lattice, not a policy that could be misconfigured.

Proposition 3.2 (Fail-Closed Default)**.** The tensor initializes every domain to PENDING. Updates require temporally valid, cryptographically verified attestations. In the absence of an attestation for domain d, $T[d]$ remains PENDING (non-passing).

An intelligent asset whose compliance state is unknown cannot act. Ignorance blocks; it does not permit. Traditional systems tend in the opposite direction: unchecked transactions proceed. Here, the default is denial.

The lattice also admits a residual operation: given current state a and target state b, the residual $a \to b = \bigvee\{c : a \wedge c \leq b\}$ computes the minimum improvement needed to reach b from a. This answers a practical question: what additional attestations must an asset obtain to become compliant for a given operation?

4 Execution

4.1 The Virtual Machine

The Smart Asset Virtual Machine (SAVM) is a stack-based bytecode interpreter: 47 opcodes across 12 categories, 32-byte words aligned with SHA-256 digests, gas metering across 18 cost classes, deterministic storage via BTreeMap, and hash-chained execution receipts. The instruction set spans stack manipulation, arithmetic, comparison, memory access, persistent storage, control flow, context queries (caller, jurisdiction, timestamp, asset identity), compliance operations, cryptography (SHA-256, Ed25519 verification, Merkle proofs), and system operations (halt, log).

4.2 The Compliance Gate

Compliance is enforced at two levels.

System level. Before any program executes, the runtime evaluates the compliance tensor for the operation's required domains. If the verdict is non-passing, execution does not begin. After execution completes, the tensor is re-evaluated to confirm that

[2]The name is conventional. The object is a function $\mathcal{D} \to \mathcal{S}$—equivalently, a 20-element array over $\mathcal{S}$.

the program's effects have not invalidated compliance. This preflight/postflight gate cannot be bypassed by a program.

Program level. Four opcodes let programs query and interact with the compliance tensor during execution:

- `TENSOR_GET`(e, j, d): read the compliance state of entity e in jurisdiction j for domain d.
- `TENSOR_SET`(e, j, d, v): write a compliance state (restricted to zone-level authority).
- `ATTEST`(payload): record an attestation reference in the execution context.
- `VERIFY_ZK`(proof, verifier): verify a zero-knowledge proof (Groth16 over BN254 or PLONK over Pallas).

The system-level gate ensures that no program can act without compliance clearance. The program-level opcodes let programs make compliance-dependent decisions within their logic—for instance, choosing a different counterparty if the first fails a sanctions check.

4.3 Receipts

Every execution produces a receipt, including failed executions. Receipts are hash-chained: each contains the digest of its predecessor. The chain records what the asset did, what it attempted and failed, which domains blocked it, and which attestations it consumed. A regulator can reconstruct the asset's complete decision history from the chain alone. Storage mutations persist across executions, giving the asset continuity—its portfolio state, performance history, and operational parameters survive between program runs.

4.4 Triggers

Triggers bind external events to bound programs:

- **Regulatory:** sanctions list update, attestation expiry, compliance state change.
- **Settlement:** payment received, custody transferred, netting cycle completed.
- **Asset:** NAV breach, portfolio concentration limit, rebalancing threshold.
- **Corridor:** migration opportunity, fee change, jurisdictional quality shift.
- **Temporal:** reporting deadline, distribution date, scheduled evaluation.

When an event matches a trigger, the runtime executes the bound program. The compliance gate applies: a non-passing verdict reverts execution and the receipt records the cause.

4.5 Negotiation

Two intelligent assets can negotiate through a two-phase atomic protocol. In the first phase, one asset proposes terms and both evaluate the proposal against their own tensors. In the second phase, if both verdicts pass, settlement is atomic: both storage states update, both receipt chains extend, and attestations cross-reference. If either verdict fails, the negotiation aborts. This is how a fund rebalances: it negotiates with counterparty assets—bonds, equity positions, debt instruments—each an intelligent asset with its own compliance tensor.

5 Composition

When an intelligent asset migrates from J_1 to J_2, the corridor $C(J_1, J_2) = (R, \varphi)$ determines what happens: domains in R are freshly evaluated by J_2; domains with a mapping through φ carry attestations from J_1; unmapped domains not in R default to PENDING.

Tensor meet is associative: $(T_1 \wedge T_2) \wedge T_3 = T_1 \wedge (T_2 \wedge T_3)$, since $\wedge = \min$ on a total order. Multi-hop composition is well-defined regardless of grouping.

Migration is not commutative. The path $J_1 \to J_2 \to J_3$ may produce a different compliance state than $J_1 \to J_3 \to J_2$, because each corridor defines its own re-evaluation set. Route choice matters: a fund should prefer paths that maximize attestation reuse and minimize fresh evaluation. Route optimization over a jurisdictional network is a well-defined graph problem.

A *mutual recognition agreement* between J_1 and J_2 for domain subset $D' \subseteq \mathcal{D}$ means neither jurisdiction requires re-evaluation for domains in D'. MRAs reduce the corridor's re-evaluation set, lowering migration cost and latency.

When exit costs approach zero, jurisdictional competition sharpens. An intelligent asset that encounters regulatory friction does not petition for reform; it migrates, and the receipt chain records exactly why. This is Hirschman's exit-voice framework applied to financial regulation: portable compliance attestations reduce exit costs from months to seconds, creating jurisdictional feedback on timescales that were previously impossible. Whether this feedback improves regulatory quality or triggers a race to the bottom is a political economy question addressed in §8.

6 Worked Example: An Autonomous PE Fund

We trace the lifecycle of a private equity fund structured as an intelligent asset.

Genesis

The fund is created in ADGM (Abu Dhabi Global Market). Its genesis document records: fund type, investment mandate (global multi-asset, $\leq$40% MENA concentration), NAV parameters. Five programs are bound: portfolio rebalancing, NAV computation, investor distribution, regulatory reporting, migration evaluation. Triggers are set: MENA concentration exceeds 40%, quarterly NAV computation, distribution dates, reporting deadlines, sanctions list updates.

ADGM's evaluator assesses all twenty domains against the current lawpack. The tensor passes. The fund is active.

Rebalancing

Three months in, MENA real estate appreciation pushes allocation to 43%. The concentration trigger fires. The rebalancing program executes:

1. Read the portfolio from storage.
2. Compute target allocation per the investment mandate.
3. Identify counterparty assets: Southeast Asian infrastructure bonds, each itself an intelligent asset.
4. For each proposed trade, call `TENSOR_GET` for sanctions, AML, KYC, securities, and cross-border domains. All must pass.
5. Initiate negotiation with each counterparty. Both sides evaluate their tensors.
6. On mutual pass, settle atomically: update portfolio in storage, extend receipt chains, cross-reference attestations.
7. On any failure, `REVERT`. The receipt records the blocking domain.

Gas metering caps execution cost. The receipt chain records every step, every decision, every outcome.

Migration

The fund's migration program identifies that MAS (Singapore) offers lower custody fees. It evaluates ADGM exit requirements: the tensor must pass all twenty domains. It does. ADGM issues an exit attestation.

The fund presents its attestation portfolio to MAS. The corridor $C(\text{ADGM}, \text{MAS})$ defines re-evaluation set $R = \{\text{Securities}, \text{Licensing}\}$—ADGM's "Specified Investment" classification does not map directly to MAS's "Capital Markets Product." MAS evaluates these two domains and issues fresh attestations. The remaining eighteen carry over. All twenty pass. MAS issues an entry attestation. The fund is now domiciled in Singapore. Its programs continue under MAS rules with the Singapore lawpack.

Sanctions Event

A beneficial owner of a portfolio company appears on a sanctions list. The sanctions trigger fires. The tensor updates: $T[\text{Sanctions}] \leftarrow \text{NONCOMPLIANT}$. By Proposition 3.1, $V(T) = \bot$. Every program is blocked. The fund cannot trade, rebalance, distribute, or migrate. The receipt records the cause. Resolution requires sanctions clearance, a fresh attestation, and a tensor update. Until then, the fund is frozen—immediately, automatically, and by the algebra, without possibility of override.

7 Comparison

7.1 Smart Contracts

Both intelligent assets and smart contracts embed executable logic in financial instruments. The difference is the relationship between the logic and authority.

A smart contract executes unconditionally: its code is its law. When the DAO was exploited in 2016 through a reentrancy vulnerability, the $60 million drain was "correct" by the code's own standard. An intelligent asset executes conditionally: the compliance tensor must pass before any action proceeds, and sovereign evaluators can revoke attestations after the fact. The asset's logic runs under law, not as law.

Smart contracts optimize for trustlessness. Intelligent assets make trust explicit and scoped: an attestation from ADGM means that ADGM's evaluator assessed compliance under a specific versioned lawpack. The receiving jurisdiction can verify the signature, inspect the lawpack, and decide whether to accept or re-evaluate. Trust is auditable, not absent.

Smart contract state is immutable absent a hard fork. Attestations expire and are revocable. When regulations change or evaluations err, the system adapts.

7.2 Traditional Compliance

Traditional compliance produces proprietary, non-portable, non-composable artifacts. Cross-jurisdictional requirements are assembled by lawyers. Three differences: (i) attestations are portable—W3C Verifiable Credentials with standard formats and verification; (ii) composition is algebraic—multi-jurisdictional requirements combine through lattice meet, not legal memoranda; (iii) evaluation rules are versioned and content-addressed—the rules under which an evaluation was performed are a matter of record.

The structural difference: intelligent assets operate autonomously. Traditional compliance is a human process measured in months per jurisdiction. When every cross-border action requires manual review, entire categories of financial instrument are uneconomic.

8 Limitations

The oracle problem. The tensor's properties—fail-closed default, sanctions absorption—are properties of the algebra, not of the world. A tensor faithfully representing incorrect evaluations is worthless. Evaluator correctness cannot be verified from within the system: cryptographic signatures prove provenance, not accuracy. The only mitigations are institutional—audit rights, cross-jurisdictional challenges, reputational accountability. This is the deepest limitation and it is inherent.

Legal force. Attestations are technically verifiable but have legal force only if sovereign authorities recognize them. This is political and legal, not technical. The system provides infrastructure that legal agreements can reference. It does not create legal force on its own.

Bootstrapping. The system requires jurisdictions to deploy evaluators and agree on corridor definitions. Small jurisdictions can adopt early but lack credibility. Large jurisdictions have credibility but no incentive to pioneer. No amount of formal elegance resolves this.

Race to the bottom. If jurisdictions compete by weakening compliance standards, the system accelerates regulatory arbitrage rather than preventing it. The sanctions hard-block provides a floor: no jurisdiction can attract assets by ignoring sanctions. Corridors can require full re-evaluation from weak jurisdictions. But fraudulent attestations cannot be detected cryptographically. The race to the bottom is a political economy problem, and we offer better tools for managing it, not a solution.

Irreducible complexity. Some compliance determinations—novel asset structures, ambiguous beneficial ownership, adversarial tax planning—require human judgment. Autonomous operation applies to the subset of compliance that is fully algorithmic. The system supports hybrid evaluation: algorithmic for routine cases, human-in-the-loop for the rest.

Agent alignment. An intelligent asset executes its bound programs. Who writes them? Who updates them when market conditions change? Who is liable when an autonomous rebalancing produces losses? These are governance questions. The formal architecture is silent on them.

Privacy. Zero-knowledge proofs can demonstrate compliance without revealing underlying data, but only for predicates expressible as arithmetic circuits. Many compliance determinations involve unstructured evidence. Graduated disclosure—different counterparties see different levels of detail—is a pragmatic compromise.

9 Conclusion

An intelligent asset carries programs, a compliance tensor, sovereign attestations, and a hash-linked execution record. The programs give it autonomy. The tensor gives it legality. The attestations give it portability. The receipt chain gives it auditability.

The tensor is a function from twenty regulatory domains to a five-element bounded lattice. Its properties—fail-closed default, sanctions non-overridability, associative composition—follow from the lattice structure and require no runtime enforcement beyond initialization. The architecture makes autonomous cross-jurisdictional financial operation formally possible.

Whether it becomes actual depends on jurisdictional adoption, legal recognition, and governance frameworks that are outside the scope of formal methods.

Implementation

The reference implementation comprises 18 Rust crates, 445 source files, 317,264 lines of code, and 8,656 test annotations. The SAVM interpreter, compliance tensor algebra, attestation issuance (Ed25519 VCs), policy compiler, and asset lifecycle state machine are production-grade. The trigger system and negotiation protocol are implemented. Migration is a stub. ZK verification (Groth16 over BN254, PLONK over Pallas) is feature-gated.

References

[1] A. O. Hirschman, *Exit, Voice, and Loyalty*, Harvard University Press, 1970.

[2] W3C, "Verifiable Credentials Data Model v2.0," W3C Recommendation, 2024.

[3] G. Birkhoff, *Lattice Theory*, 3rd ed., AMS, 1967.

[4] B. A. Davey and H. A. Priestley, *Introduction to Lattices and Order*, 2nd ed., CUP, 2002.

[5] J. H. Saltzer, D. P. Reed, D. D. Clark, "End-to-end arguments in system design," *ACM TOCS*, 2(4):277–288, 1984.

[6] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008.

[7] V. Buterin, “Ethereum White Paper,” 2014.

[8] C. M. Tiebout, “A Pure Theory of Local Expenditures,” *JPE*, 64(5):416–424, 1956.

[9] G. C. Necula, “Proof-carrying code,” *POPL*, pp. 106–119, 1997.